

Overview of DBSCAN

Subject: DBSCAN

1.

Parameter estimation Every data mining task has the problem of parameters. Every parameter influences the algorithm in specific ways. For DBSCAN, the parameters ϵ and minPts are needed. The parameters must be specified by the user. Ideally, the value of ϵ is given by the problem to solve (e.g. a physical distance), and minPts is then the desired minimum cluster size.

2.

Original query-based algorithm DBSCAN requires two parameters: ϵ (eps) and the minimum number of points required to form a dense region (minPts). It starts with an arbitrary starting point that has not been visited. This point's ϵ -neighborhood is retrieved, and if it contains sufficiently many points, a cluster is started. Otherwise, the point is labeled as noise. Note that this point might later be found in a sufficiently sized ϵ -environment of a different point and hence be made part of a cluster. If a point is found to be a dense part of a cluster, its ϵ -neighborhood is also part of that cluster. Hence, all points that are found within the ϵ -neighborhood are added, as is their own ϵ -neighborhood when they are also dense. This process continues until the density-connected cluster is completely found. Then, a new unvisited point is retrieved and processed, leading to the discovery of a further cluster or noise. DBSCAN can be used with any distance function (as well as similarity functions or other predicates). The distance function (dist) can therefore be seen as an additional parameter. The algorithm can be expressed in pseudocode as follows:

3.

Relationship to spectral clustering A spectral implementation of DBSCAN is related to spectral clustering in the trivial case of determining connected graph components — the optimal clusters with no edges cut. However, it can be computationally intensive, up to $O(n^3)$. Additionally, one has to choose the number of eigenvectors to compute. For performance reasons, the original DBSCAN algorithm remains preferable to its spectral implementation.

4.

```
RangeQuery(DB, distFunc, Q, eps) {  
  Neighbors N := empty list  
  for each point P in database DB {  
    /* Scan all points in the database */  
    if distFunc(Q, P) ≤ eps then {  
      /* Compute distance and check epsilon */  
      N := N ∪ {P}  
    }  
  }  
  /* Add to result */  
  return N  
}
```

5.

MinPts: As a rule of thumb, a minimum minPts can be derived from the number of dimensions D in the data set, as $\text{minPts} \geq D + 1$. The low value of $\text{minPts} = 1$ does not make sense, as then every point is a core point by definition. With $\text{minPts} \leq 2$, the result will be the same as of hierarchical clustering with the single link metric, with the dendrogram cut at height ϵ . Therefore, minPts must be chosen at least 3. However, larger values are usually better for data sets with noise and will yield more significant clusters. As a rule of thumb, $\text{minPts} = 2 \cdot \text{dim}$ can be used, but it may be necessary to choose larger values for very large data, for noisy data or for data that contains many duplicates. ϵ : The value for ϵ

can then be chosen by using a k-distance graph, plotting the distance to the $k = \text{minPts} - 1$ nearest neighbor ordered from the largest to the smallest value. Good values of ϵ are where this plot shows an "elbow": if ϵ is chosen much too small, a large part of the data will not be clustered; whereas for a too high value of ϵ , clusters will merge and the majority of objects will be in the same cluster. In general, small values of ϵ are preferable, and as a rule of thumb only a small fraction of points should be within this distance of each other. Alternatively, an OPTICS plot can be used to choose ϵ , but then the OPTICS algorithm itself can be used to cluster the data. Distance function: The choice of distance function is tightly coupled to the choice of ϵ , and has a major impact on the results. In general, it will be necessary to first identify a reasonable measure of similarity for the data set, before the parameter ϵ can be chosen. There is no estimation for this parameter, but the distance functions needs to be chosen appropriately for the data set. For example, on geographic data, the great-circle distance is often a good choice. OPTICS can be seen as a generalization of DBSCAN that replaces the ϵ parameter with a maximum value that mostly affects performance. MinPts then essentially becomes the minimum cluster size to find. While the algorithm is much easier to parameterize than DBSCAN, the results are a bit more difficult to use, as it will usually produce a hierarchical clustering instead of the simple data partitioning that DBSCAN produces. Recently, one of the original authors of DBSCAN has revisited DBSCAN and OPTICS, and published a refined version of hierarchical DBSCAN (HDBSCAN*), which no longer has the notion of border points. Instead, only the core points form the cluster.

6.

Find the points in the ϵ (eps) neighborhood of every point, and identify the core points with more than minPts neighbors. Find the connected components of core points on the neighbor graph, ignoring all non-core points. Assign each non-core point to a nearby cluster if the cluster is an ϵ (eps) neighbor, otherwise assign it to noise. A naive implementation of this requires storing the neighborhoods in step 1, thus requiring substantial memory. The original DBSCAN algorithm does not require this by performing these steps for one point at a time.

7.

Complexity DBSCAN visits each point of the database, possibly multiple times (e.g., as candidates to different clusters). For practical considerations, however, the time complexity is mostly governed by the number of regionQuery invocations. DBSCAN executes exactly one such query for each point, and if an indexing structure is used that executes a neighborhood query in $O(\log n)$, an overall average runtime complexity of $O(n \log n)$ is obtained (if parameter ϵ is chosen in a meaningful way, i.e. such that on average only $O(\log n)$ points are returned). Without the use of an accelerating index structure, or on degenerated data (e.g. all points within a distance less than ϵ), the worst case run time complexity remains $O(n^2)$. The $\binom{n}{2} = \frac{n(n-1)}{2}$ -sized upper triangle of the distance matrix can be materialized to avoid distance recomputations, but this needs $O(n^2)$ memory, whereas a non-matrix based implementation of DBSCAN only needs $O(n)$ memory.